

and then the y - z path in T . The length of this cycle is $(j - i) + 1 + (j - i)$, adding the length of its three parts separately; this is equal to $2(j - i) + 1$, which is an odd number. ■

3.5 Connectivity in Directed Graphs

Thus far, we have been looking at problems on undirected graphs; we now consider the extent to which these ideas carry over to the case of directed graphs.

Recall that in a directed graph, the edge (u, v) has a direction: it goes from u to v . In this way, the relationship between u and v is asymmetric, and this has qualitative effects on the structure of the resulting graph. In Section 3.1, for example, we discussed the World Wide Web as an instance of a large, complex directed graph whose nodes are pages and whose edges are hyperlinks. The act of browsing the Web is based on following a sequence of edges in this directed graph; and the directionality is crucial, since it's not generally possible to browse "backwards" by following hyperlinks in the reverse direction.

At the same time, a number of basic definitions and algorithms have natural analogues in the directed case. This includes the adjacency list representation and graph search algorithms such as BFS and DFS. We now discuss these in turn.

Representing Directed Graphs

In order to represent a directed graph for purposes of designing algorithms, we use a version of the adjacency list representation that we employed for undirected graphs. Now, instead of each node having a single list of neighbors, each node has two lists associated with it: one list consists of nodes *to which* it has edges, and a second list consists of nodes *from which* it has edges. Thus an algorithm that is currently looking at a node u can read off the nodes reachable by going one step forward on a directed edge, as well as the nodes that would be reachable if one went one step in the reverse direction on an edge from u .

The Graph Search Algorithms

Breadth-first search and depth-first search are almost the same in directed graphs as they are in undirected graphs. We will focus here on BFS. We start at a node s , define a first layer of nodes to consist of all those to which s has an edge, define a second layer to consist of all additional nodes to which these first-layer nodes have an edge, and so forth. In this way, we discover nodes layer by layer as they are reached in this outward search from s , and the nodes in layer j are precisely those for which the shortest path *from* s has exactly j edges. As in the undirected case, this algorithm performs at most constant work for each node and edge, resulting in a running time of $O(m + n)$.

It is important to understand what this directed version of BFS is computing. In directed graphs, it is possible for a node s to have a path to a node t even though t has no path to s ; and what directed BFS is computing is the set of all nodes t with the property that s has a path to t . Such nodes may or may not have paths back to s .

There is a natural analogue of depth-first search as well, which also runs in linear time and computes the same set of nodes. It is again a recursive procedure that tries to explore as deeply as possible, in this case only following edges according to their inherent direction. Thus, when DFS is at a node u , it recursively launches a depth-first search, in order, for each node to which u has an edge.

Suppose that, for a given node s , we wanted the set of nodes with paths to s , rather than the set of nodes to which s has paths. An easy way to do this would be to define a new directed graph, G^{rev} , that we obtain from G simply by reversing the direction of every edge. We could then run BFS or DFS in G^{rev} ; a node has a path *from* s in G^{rev} if and only if it has a path *to* s in G .

Strong Connectivity

Recall that a directed graph is *strongly connected* if, for every two nodes u and v , there is a path from u to v and a path from v to u . It's worth also formulating some terminology for the property at the heart of this definition; let's say that two nodes u and v in a directed graph are *mutually reachable* if there is a path from u to v and also a path from v to u . (So a graph is strongly connected if every pair of nodes is mutually reachable.)

Mutual reachability has a number of nice properties, many of them stemming from the following simple fact.

(3.16) *If u and v are mutually reachable, and v and w are mutually reachable, then u and w are mutually reachable.*

Proof. To construct a path from u to w , we first go from u to v (along the path guaranteed by the mutual reachability of u and v), and then on from v to w (along the path guaranteed by the mutual reachability of v and w). To construct a path from w to u , we just reverse this reasoning: we first go from w to v (along the path guaranteed by the mutual reachability of v and w), and then on from v to u (along the path guaranteed by the mutual reachability of u and v). ■

There is a simple linear-time algorithm to test if a directed graph is strongly connected, implicitly based on (3.16). We pick any node s and run BFS in G starting from s . We then also run BFS starting from s in G^{rev} . Now, if one of these two searches fails to reach every node, then clearly G is not strongly connected. But suppose we find that s has a path to every node, and that

every node has a path to s . Then s and v are mutually reachable for every v , and so it follows that *every* two nodes u and v are mutually reachable: s and u are mutually reachable, and s and v are mutually reachable, so by (3.16) we also have that u and v are mutually reachable.

By analogy with connected components in an undirected graph, we can define the *strong component* containing a node s in a directed graph to be the set of all v such that s and v are mutually reachable. If one thinks about it, the algorithm in the previous paragraph is really computing the strong component containing s : we run BFS starting from s both in G and in G^{rev} ; the set of nodes reached by *both* searches is the set of nodes with paths to *and* from s , and hence this set is the strong component containing s .

There are further similarities between the notion of connected components in undirected graphs and strong components in directed graphs. Recall that connected components naturally partitioned the graph, since any two were either identical or disjoint. Strong components have this property as well, and for essentially the same reason, based on (3.16).

(3.17) *For any two nodes s and t in a directed graph, their strong components are either identical or disjoint.*

Proof. Consider any two nodes s and t that are mutually reachable; we claim that the strong components containing s and t are identical. Indeed, for any node v , if s and v are mutually reachable, then by (3.16), t and v are mutually reachable as well. Similarly, if t and v are mutually reachable, then again by (3.16), s and v are mutually reachable.

On the other hand, if s and t are not mutually reachable, then there cannot be a node v that is in the strong component of each. For if there were such a node v , then s and v would be mutually reachable, and v and t would be mutually reachable, so from (3.16) it would follow that s and t were mutually reachable. ■

In fact, although we will not discuss the details of this here, with more work it is possible to compute the strong components for all nodes in a total time of $O(m + n)$.

3.6 Directed Acyclic Graphs and Topological Ordering

If an undirected graph has no cycles, then it has an extremely simple structure: each of its connected components is a tree. But it is possible for a directed graph to have no (directed) cycles and still have a very rich structure. For example, such graphs can have a large number of edges: if we start with the node